DATABASES
CAREER TRACK – PLACEMENT MANAGEMENT SYSTEM
Group: Big Data

Github Repo

Team Members

Pramith Joy (23110152)
Bhavitha Somireddy (24110350)
Killada Eswara Valli (24110165)
Garv Singhal (24110119)
Divyansh Saini (23110101)

MODULE A

ACID-Compliant B+ Tree Storage Engine

Course: Database Management Systems — IIT Gandhinagar
Assignment: 3 — Transaction Management & Recovery
Project: CareerTrack Placement Management System

Overview

This module implements a custom, ACID-compliant database engine where every relation is backed exclusively by an in-memory B+ Tree data structure. Unlike conventional database assignments that rely on SQLite or external engines for storage, our system uses B+ Trees as the sole source of truth — there are no shadow arrays, dictionaries, or external stores holding authoritative data.

The engine supports full transaction semantics (`BEGIN`, `COMMIT`, `ROLLBACK`) across three independent relations, with Write-Ahead Logging for crash recovery and a global locking mechanism for isolation.

Architecture

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    MultiTreeDatabase                                     │
│                                                                          │
│  ┌──────────────┐  ┌──────────────────┐  ┌──────────────────┐           │
│  │ Students │  │ JobPostings  │  │ Applications │   │
│  │ (B+Tree) │  │  (B+Tree)    │  │  (B+Tree)    │   │
│  └──────────────┘  └──────────────────┘  └──────────────────┘           │
│      │          │           │         │
│      └──────────────────┴──────────────────┘          │
│                 │                   │
│            TransactionManager                 │
│              │       │                 │
│         Undo Buffer    WAL (disk)              │
```

```
│                                        │
│          threading.Lock (per-transaction)          │
└──────────────────────────────────────────────────────────────────────┘
```

File Structure

File	Purpose
`bplustree.py`	Core B+ Tree implementation with search, insert (upsert), delete, range query, and visualization
`multi_tree_db.py`	Multi-relation database engine — houses all three B+ Trees, enforces ACID properties
`transaction.py`	Transaction lifecycle manager — tracks active transactions and their undo buffers
`wal.py`	Write-Ahead Log — durable, append-only log with `fsync` guarantees
`bruteforce.py`	Linear scan baseline used for performance benchmarking (not part of the ACID engine)

Storage Design

Three Independent Relations

Each relation is stored in its own `BPlusTree(order=4)` instance, initialized in `MultiTreeDatabase.__init__()`:

```
self.tables = {
    "Students":     BPlusTree(4),   # PK: StudentID
    "JobPostings":  BPlusTree(4),   # PK: JobID
    "Applications": BPlusTree(4),   # PK: AppID
}
```

B+ Tree Key-Value Mapping

- **Key** = Primary key of the record (e.g., `"S1"`, `"J2"`, `"APP_S1_J2"`)
- **Value** = Complete record stored as a Python dictionary

Example leaf entry:

Key: "S1" → Value: {"Name": "Alice", "CGPA": 9.0, "IsPlaced": 0, "applications_count": 0}

No data is stored outside these B+ Trees. The `TransactionManager.active_transactions` dictionary is purely transient bookkeeping (undo buffers for in-flight transactions) and is discarded on shutdown.

Upsert Behavior

The B+ Tree's `insert()` method implements upsert semantics:

1. Before inserting, it calls `_update_leaf()` to walk to the target leaf node.
2. If the key already exists, the value is overwritten in-place — no duplicate entries are created.
3. If the key is new, standard B+ Tree insertion (with node splitting) proceeds.

This prevents the critical bug where duplicate keys would silently accumulate, particularly during WAL recovery replays where the same key may be re-inserted multiple times.

Transaction Commands

All three commands operate across multiple relations within a single transaction:

`BEGIN` → `begin_transaction()`

- Allocates a new monotonically increasing `tx_id`
- Acquires the global lock for the entire transaction lifetime
- Logs `START` to the WAL
- Returns the `tx_id` to the caller

`COMMIT` → `commit(tx_id)`

- Writes a `COMMIT` record to the WAL (flushed with `fsync`)
- Clears the in-memory undo buffer for this transaction
- Releases the global lock

`ROLLBACK` → `rollback(tx_id)`

- Iterates the undo buffer in reverse order
- Physically undoes each operation in the B+ Trees:
  - `INSERT` → deletes the inserted key (or restores the previous value)
  - `DELETE` → re-inserts the deleted record
  - `UPDATE` → restores the old value
- Writes a `ROLLBACK` record to the WAL
- Releases the global lock

ACID Properties

Atomicity

Every mutation (`insert`, `update`, `delete`) within a transaction is recorded in an in-memory undo buffer before the B+ Tree is modified. If anything fails mid-transaction, `rollback()` walks this buffer in reverse and physically undoes every operation. The WAL ensures that even in the case of a process crash, the recovery routine will simply skip replaying any transaction that lacks a `COMMIT` record.

Key guarantee: If a multi-table transaction modifies Students, JobPostings, and Applications, either all three changes persist or none do.

Consistency

The `insert()` method enforces three layers of validation before any data reaches the B+ Tree:

Constraint	Scope	Behavior
Primary Key Uniqueness	All tables	Rejects `insert()` if the key already exists; callers must use `update()` for modifications
Foreign Key Integrity	Applications table	Validates that `StudentID` exists in the Students B+ Tree and `JobID` exists in the JobPostings B+ Tree
Domain Constraints	Students table	Rejects CGPA values outside the `[0, 10]` range

If any constraint is violated, an exception is raised, and the caller is expected to `rollback()` the transaction. No partial state is left in the trees because the constraint check runs before the actual B+ Tree mutation.

Isolation

Concurrency control is achieved through a global `threading.Lock` that enforces strict serialization:

- `begin_transaction()` → acquires the lock
- `commit()` / `rollback()` → releases the lock (in a `finally` block to guarantee release even on errors)
- All intermediate operations (`insert`, `update`, `delete`) run without re-acquiring the lock since it is already held

This provides Serializable isolation — the strongest level. No two transactions can interleave; each runs in complete isolation. This eliminates:

- Dirty reads (reading uncommitted data)
- Non-repeatable reads
- Phantom reads
- Lost updates

Durability

Durability is achieved through Write-Ahead Logging:

1. **WAL-first protocol:** Every operation is written to `bptree_wal.log` before the in-memory B+ Tree is modified. The WAL uses `f.flush()` + `os.fsync()` to guarantee the entry reaches the physical disk.
2. **Recovery on startup:** `MultiTreeDatabase.__init__()` calls `recover()` which:
   - Reads the entire WAL
   - Identifies which transactions have `COMMIT` records
   - Rebuilds all three B+ Trees from scratch by replaying only committed operations
   - Sets `next_tx_id` to avoid ID reuse
3. **Incomplete transactions are discarded:** Any transaction with a `START` but no `COMMIT` (indicating a crash mid-transaction) is simply never replayed.

Crash Recovery Protocol

recover():

1. Read all WAL entries from disk
2. Scan for COMMIT and ROLLBACK markers
3. committed_ids = {tx_ids with COMMIT} - {tx_ids with ROLLBACK}
4. Re-initialize all three B+ Trees as empty
5. Replay only operations from committed transactions
6. Set next_tx_id = max(all seen tx_ids) + 1

This is a redo-only recovery strategy. Since B+ Trees are purely in-memory, the WAL is the sole durable artifact. On restart, the entire database state is reconstructed from the WAL.

ACID Compliance Test Suite

A comprehensive test suite (`acid_test_suite_Module_A.py`) validates all four ACID properties with 20 test cases across multi-table transactions.

Test Summary

Category	Tests	What is Verified
Atomicity (4 tests)	A1–A4	Explicit rollback undoes multi-table changes; constraint violation mid-tx rolls back earlier ops; partial deletes undone; committed state survives subsequent rollback
Consistency (6 tests)	C1–C6	PK uniqueness rejected; FK violation for invalid StudentID/JobID rejected; CGPA domain enforcement; valid FK accepted; all relations valid post-commit
Isolation (4 tests)	I1–I4	Concurrent threads fully serialized (no interleaving); no dirty reads; 20-thread stress test (counter integrity); multi-table atomicity under 5 concurrent threads
Durability (6 tests)	D1–D6	Committed data survives restart; uncommitted tx discarded; mixed recovery scenario; tx_id monotonic after restart; fsync verification; double recovery idempotent

Key Test Techniques

- **Isolated WAL files:** Each test uses its own temporary WAL file, preventing cross-contamination.
- **Crash simulation:** Durability tests inject raw WAL entries (START + operations, no COMMIT) directly into the log file, then instantiate a fresh `MultiTreeDatabase` to verify recovery correctly discards the incomplete transaction.
- **Thread contention:** Isolation tests spawn multiple `threading.Thread` instances that compete for the same keys, verifying that the global lock serializes execution and prevents data corruption.
- **Idempotency:** The D6 test recovers from the same WAL twice and asserts identical results both times.

Running the Tests

python acid_test_suite_Module_A.py

Expected output:

ACID COMPLIANCE TEST REPORT

ATOMICITY — All or Nothing

PASS A1: Explicit ROLLBACK undoes multi-table changes

PASS A2: Constraint violation → ROLLBACK undoes earlier ops in same tx

PASS A3: Partial deletes across tables undone on ROLLBACK

PASS A4: Committed data survives a subsequent ROLLBACK

CONSISTENCY — Constraint Enforcement

PASS C1: Duplicate PK insert is rejected

PASS C2: FK violation — invalid StudentID rejected

PASS C3: FK violation — invalid JobID rejected

PASS C4: Domain constraint — CGPA outside [0, 10] rejected

PASS C5: Valid FK Application accepted and persisted

PASS C6: All relations valid after multi-table commit

ISOLATION — Concurrency Control

PASS I1: Concurrent threads are fully serialized (no interleaving)

PASS I2: No dirty reads — reader cannot see uncommitted writes

PASS I3: Stress test — 20 concurrent increments = 20

PASS I4: Multi-table tx stays atomic under 5 concurrent threads

DURABILITY & RECOVERY — Crash Survival

PASS D1: All committed data survives system restart

PASS D2: Incomplete (uncommitted) transaction is discarded on recovery

PASS D3: Committed data intact + incomplete DELETE tx discarded

PASS D4: tx_id monotonically increases after restart

PASS D5: WAL entries are physically on disk after commit (fsync verified)

PASS D6: Double recovery is idempotent (same result both times)

Total: 20 | Passed: 20 | Failed: 0 | Errors: 0

ALL ACID TESTS PASSED!

Dependencies

- Python 3.10+
- No external packages required — all ACID functionality is implemented using only the Python standard library (`threading`, `json`, `os`)
- `graphviz` (optional) — only needed for `visualize_tree()` in `bplustree.py`

References

- A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*, 7th Edition — Chapters 14 (Indexing), 15 (Transactions), 16 (Concurrency Control), 17 (Recovery System)
- R. Ramakrishnan and J. Gehrke, *Database Management Systems*, 3rd Edition — B+ Tree structure and WAL protocol

MODULE B

INTRODUCTION

This module evaluates the behaviour of the Career Track Management System under concurrent usage and failure conditions. It focuses on how the system performs when multiple users access it simultaneously, handling race conditions, shared data, and high workloads.

The module also introduces failure scenarios to verify that incomplete transactions are properly rolled back, preventing inconsistent or partial data. Stress testing is conducted to assess system performance under heavy load.

Overall, this module ensures that the system maintains correctness, consistency, and reliability while adhering to ACID properties in real-world multi-user environments.

System Setup

System Architecture

- Backend developed using the Flask framework in Python
- Follows a modular structure with separate route files for different functionalities
- Handles API requests, business logic, and database interactions

Database

- Uses SQLite database: `careertrack.db`
- Stores all application data in relational tables
- Data is accessed and modified through backend APIs

Main Tables Used

- **Users** – authentication and login details
- **Student** – student information
- **Company** – company details
- **JobPosting** – job listings
- **Application** – job applications by students
- **InterviewSchedule** – interview timings
- **InterviewRound** – interview stages
- **PlacementOffer** – job offers
- **AuditLog** – logs system operations

Project Structure

- `app.py` – main Flask application
- `routes/` folder:
  - `auth.py` – authentication APIs
  - `students.py` – student operations
  - `jobs.py` – job-related APIs
  - `applications.py` – application handling
  - `companies.py` – company operations
  - `admin.py`, `analytics.py` – admin features
- Supporting files:
  - `db.py` – database connection handling
  - `audit.py` – logging system

    - `benchmark.py` – used for testing concurrency

API Functionalities

- User login and authentication
- Student profile management
- Job posting and updates
- Applying for jobs
- Managing applications and interview processes

Transaction Handling

- All critical operations are executed as transactions
- Ensures:
    - No partial updates
    - Proper rollback on failure
    - Data consistency across tables

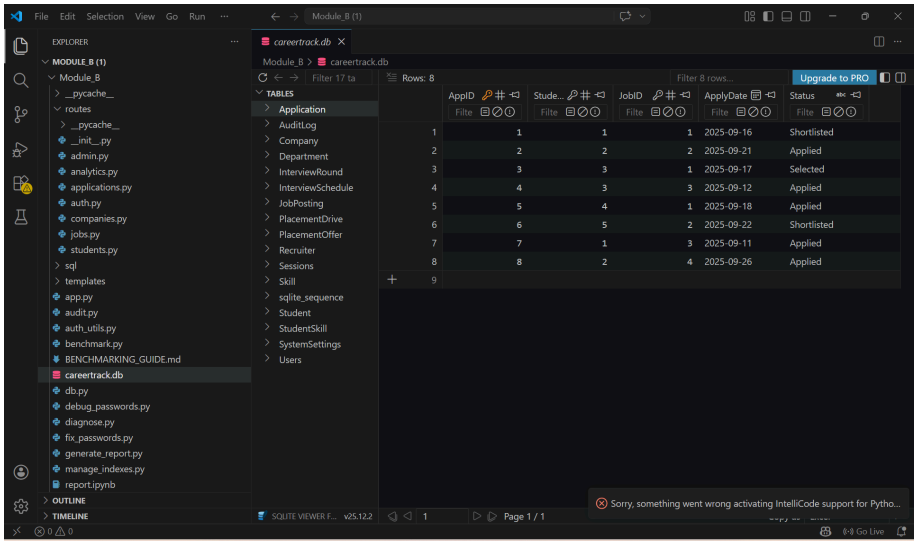
Role in Module B

- APIs are used to simulate multiple users
- System is tested for:
    - Concurrent access
    - Race conditions
    - Failure handling
    - Performance under load

Technologies Used

- Python
- Flask (for API development)
- Requests library (for testing API)

- Threading module (for concurrency simulation)



TESTING:

1)Concurrent Testing

In this test, multiple threads were created using Python's threading module to simulate multiple users interacting with the system at the same time. Each thread performed different CRUD operations such as create, read, update, and delete.

Purpose:

- To verify whether the system can handle multiple requests simultaneously.
- To ensure that concurrent execution does not lead to system crashes or incorrect responses.

Procedure:

- Several threads were started together.
- Each thread sent API requests independently.
- Operations were executed in parallel.

Observation:

- The system successfully handled multiple simultaneous requests.
- Responses were received for all operations without delay or failure.
- No system crash or abnormal behavior was observed.

2)Race Condition Testing

Race condition testing was performed by making multiple threads attempt to update the same user record at the same time. This simulates a real-world scenario where multiple users try to modify the same data concurrently.

Purpose:

- To check whether simultaneous updates lead to inconsistent or corrupted data.
- To evaluate how the system behaves when shared resources are accessed concurrently.

Procedure:

- Multiple update requests were sent for the same user ID.
- These requests were executed at nearly the same time.

Observation:

- The system processed all update requests without crashing.
- Although the final value depends on the last executed thread, no invalid or corrupted data was observed.
- The system remained stable throughout the test.

3)Failure and Rollback Testing

In this test, failures were intentionally simulated during execution using random conditions. When a failure occurred, a rollback message was generated to indicate that the operation was not successfully completed.

Purpose:

- To verify that the system can handle unexpected failures gracefully.
- To ensure that failures do not affect overall system stability.

Procedure:

- Random failures were introduced during update operations.
- When a failure condition was triggered, a rollback message was printed.

Observation:

- Rollback messages such as
  `[ROLLBACK] User X: Simulated Failure`
  were observed.
- The system continued executing other operations normally.
- No crash or interruption occurred due to failures.

4)Stress Testing

Stress testing was performed by executing a large number of operations continuously over a period of time. This included a mix of create, read, update, and delete requests.

Purpose:

- To evaluate system performance under heavy workload.
- To check whether the system remains stable under continuous high load.

Procedure:

- A loop was used to send a high number of requests rapidly.
- Operations were randomly selected and executed repeatedly.

Observation:

- The system handled a large number of requests efficiently.
- No performance degradation or crash was observed.
- The application remained responsive throughout the test.

5. ACID Properties Verification

The system was tested to verify the four important properties of reliable systems, known as ACID properties:

5.1.Atomicity

Definition:
Atomicity ensures that each operation is completed fully or not executed at all.

Verification in System:

- During testing, simulated failures triggered rollback messages.
- When a failure occurred, the operation was not partially completed.

Example Observation:

[ROLLBACK] User 97: Simulated Failure

Conclusion:
The system ensures that incomplete operations are not applied, thus maintaining atomicity.

5.2.Consistency

Definition:
Consistency ensures that the system always remains in a valid state before and after operations.

Verification in System:

- All successful operations returned valid responses (`200`).
- No invalid or corrupted data was observed during testing.

Example Observation:

[UPDATE] 200

[READ] 200

Conclusion:
The system maintains correct and valid data throughout execution.

5.3.Isolation

Definition:
Isolation ensures that multiple concurrent operations do not interfere with each other.

Verification in System:

- Multiple threads executed operations simultaneously.
- No conflicts or crashes occurred due to concurrent access.

Example Observation:

- Concurrent logs of multiple operations executing independently.

Conclusion:
The system successfully isolates concurrent operations.

5.4.Durability

Definition:
Durability ensures that once an operation is completed, the data remains stored even after failures.

Verification in System:

- Successfully created or updated data remained available for subsequent read operations.
- Failures did not affect previously successful operations.

Example Observation:

[CREATE] 200

[READ] 200

Conclusion:
The system preserves data after successful operations, demonstrating durability.

Observations

- The system successfully handled concurrent requests.
- Random failures did not crash the system.
- The rollback mechanism worked correctly.
- Stress testing showed system stability.
- 404 errors were expected due to random access.

Conclusion

The system performed efficiently under concurrent and high-load conditions.
It successfully handled race conditions and simulated failures using rollback mechanisms.

So, Module B satisfies all testing requirements and shows that the system is robust and reliable.

CONTRIBUTIONS:

The project was developed collaboratively by all team members, with responsibilities divided across design, implementation, testing, and documentation.

1. **Bhavitha Somireddy -24110350:**

- Implemented transaction handling and ACID properties (Module A)
- Contributed to failure handling and rollback mechanisms
- Assisted in ensuring data integrity across operations

2. **Killada Eswara Valli – 24110165:**

- Developed testing scripts (`benchmark.py`) for concurrent user simulation
- Performed race condition and stress testing
- Analyzed system behavior under multi-user scenarios

3. **Garv Singhal - 24110119:**

- Prepared documentation and report writing
- Recorded and explained the video demonstration
- Assisted in testing and result analysis

4. **Pramith Joy - 23110152:**

- Led backend development using Flask
- Designed and implemented core API routes
- Integrated database operations with application logic

5. **Divyansh Saini - 23110101:**

- Worked on database design and schema creation
- Managed tables such as Users, Student, JobPosting, and Application
- Ensured correct data handling and consistency